# Todo Danza: checklist de calidad (Tipos de datos básicos, clases, métodos, atributos y variables)

## A. Rutinas o métodos

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **A-01** | **Razón de existencia** | Cada método tiene un porqué es implementado. | Sí | Se eliminó la clase Catalogo al identificar que no representaba una entidad persistente con estado propio dentro del dominio, sino una operación de consulta. En la nueva arquitectura, cada método responde a una responsabilidad bien delimitada y la consulta del catálogo está en la capa de servicios. |
| **A-02** | **Propósito bien definido** | Cada método tiene un propósito bien definido. | Sí | Cada método hace una sola cosa y fueron analizados a detalle para asegurarse. |
| **A-03** | **Nombre bien definido** | El nombre del método dice exactamente lo que hace. | Sí | Los nombres de los métodos expresan claramente la acción y el concepto del negocio que representan. Esto permite entender exactamente qué hace cada función sin necesidad de revisar su código interno. |
| **A-04** | **Nombres con verbos** | Los nombres de los métodos usan verbos. | Sí | Todos empiezan con verbo: calcular, determinar, registrar, reservar, activar, listar, buscar, etc. |
| **A-05** | **Poca dependencia** | El método depende poco del resto del programa. | Sí | Los métodos del dominio usan sus propios datos. Cada servicio depende solo de los DAO que necesita. |
| **A-06** | **Parámetros con sentido** | Los parámetros dentro del método tienen sentido en conjunto. | Sí | Los parámetros son coherentes entre sí, como (idCliente, tipoPrecio) o (nombre, categoria, talla, color). |
| **A-07** | **Orden lógico y constante** | Los parámetros siguen un orden lógico y siempre el mismo. | Sí | Se respeta un orden constante y lógico en los parámetros. La búsqueda sigue el estándar nombre -> categoría -> talla -> color, y los métodos registrarVenta y registrarApartado utilizan la misma firma y secuencia conceptual de parámetros. |

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **A-08** | **Pocos parámetros** | El método tiene pocos parámetros, de preferencia no más de 7. | Sí | El máximo de parámetros se encuentra en registrarVariante, donde hay 6 de ellos lo que se encuentra por debajo del límite recomendado. |
| **A-09** | **Partes separadas** | Las partes que se pueden separar en su propio método están separadas. | Sí | Cada cálculo y regla del sistema tiene su propio método, así se evita tener funciones gigantes o sobrecargadas, asegurando que cada una se enfoque en hacer una sola tarea bien definida. |
| **A-10** | **Longitud según lo que hace** | La longitud del método depende de lo que hace. | N/A | El diseño aún no tiene implementación |
| **A-11** | **Parámetros usados** | Se usan todos los parámetros del método. | N/A | Se verifica al implementar |
| **A-12** | **Valor devuelto válido** | Si es una función, devuelve un valor válido en todos los casos. | N/A | Se verifica al implementar |
| **A-13** | **Convención para operaciones parecidas** | Las operaciones parecidas se nombran con la misma convención. | Sí | El sistema utiliza los mismos verbos para acciones similares en todas las clases: calcular para operaciones matemáticas, determinar para evaluar estados y registrar para guardar nuevas operaciones. Esto hace que el código sea predecible y fácil de leer. |

## B. Variables

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **B-01** | **Parámetros validados** | Los parámetros de entrada de los métodos se validan cuando es necesario. | Si | Las validaciones de los requisitos tienen un lugar definido antes de guardar los datos. Los datos obligatorios del producto se revisan en registrarProducto (RF-07), y los costos, precios y duplicados en registrarVariante (RF-08). La información de contacto se valida en registrarCliente (RF-26). Los ajustes de inventario y la sobreventa se controlan en Variante, mediante ajustarExistencias, reservar y descontarVenta (RF-12 y RF-15). Aún falta añadir las validaciones de modificación (RF-10) que se asignarán posteriormente. |
| **B-02** | **Declaración cerca del uso** | Las variables se declaran cerca de donde se usan por primera vez. | N/A | Se verifica en la implementación |
| **B-03** | **Inicialización antes de usarse** | Las variables se inicializan antes de usarse. | N/A | Se verifica en la implementación |
| **B-04** | **Contadores y acumuladores** | Los contadores y acumuladores se inicializan y reinician bien. | N/A | Se verifica en la implementación |
| **B-05** | **Alcance mínimo** | Cada variable tiene el menor alcance posible. | N/A | Se verifica en la implementación |
| **B-06** | **Usos cercanos** | Los usos de una variable están cerca unos de otros. | N/A | Se verifica en la implementación |
| **B-07** | **Vida corta** | Las variables no viven más tiempo del necesario. | N/A | Se verifica en la implementación |
| **B-08** | **Sin variables globales** | Se evitan las variables globales innecesarias. | Sí | No existen variables globales ni atributos estáticos compartidos, lo que evita que el estado del programa pueda ser alterado de forma imprevista desde cualquier parte del sistema. |

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **B-09** | **Sin variables sin usar** | No hay variables sin usar. | N/A | Se verifica en la implementación |
| **B-10** | **Propósito claro** | Cada variable tiene un propósito claro. | Sí | Los atributos están nombrados y definidos de forma explícita, permitiendo entender su función exacta en la clase sin generar ambigüedades sobre los datos que almacenan. |
| **B-11** | **Sin reutilización** | Una variable no se reutiliza para cosas distintas. | Sí | Cada atributo mantiene un solo significado, ninguna variable se “recicla” para guardar datos distintos en diferentes momentos. |
| **B-12** | **Valores que cambian en configuración** | Los valores que pueden cambiar se leen de una configuración y no se escriben en el código. | Sí | Los datos cambiantes de la tienda no están fijos en el código. El umbral de pocas piezas y el número de WhatsApp se gestionan a través de ConfiguracionNegocio y ConfiguracionDAO, permitiendo que métodos como determinarDisponibilidad reciban estos valores como parámetros. |

## C. Nombres de variables

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **C-01** | **Nombres adecuados** | Los nombres dicen bien lo que representan. | Sí | Los nombres describen directamente el dato que almacenan, como precioMayoreo, existenciasFisicas o contrasenaHash, haciendo evidente su función sin necesidad de explicaciones extra. |
| **C-02** | **Conceptos del problema** | Los nombres usan conceptos del problema, no detalles técnicos. | Sí | Se utilizan términos propios de la tienda y del dominio del negocio (como apartado, mayoreo y talla), evitando usar tecnicismos en el modelo. |
| **C-03** | **Longitud adecuada** | La longitud de los nombres es adecuada. | Sí | Los nombres se escriben completos y son fáciles de leer, prefiriendo la claridad antes que recortar palabras. |
| **C-04** | **Sin nombres vagos** | Se evitan los nombres vagos o poco entendibles. | Sí | Se evitan términos genéricos o ambiguos como dato, temp u objeto. Cada nombre explica con precisión qué representa. |
| **C-05** | **Índices descriptivos** | Los índices de ciclos anidados o complejos tienen nombres descriptivos. | N/A | No aplica en un diagrama de clases. |
| **C-06** | **Variables temporales con sentido** | Las variables temporales tienen nombres con sentido cuando su propósito no es obvio. | N/A | No aplica en un diagrama de clases. |
| **C-07** | **Booleanas legibles** | Las variables booleanas se leen bien cuando valen verdadero. | Sí | Los atributos booleanos (como activo) están redactados en afirmativo, haciendo que su significado sea claro e intuitivo cuando su valor es verdadero. |
| **C-08** | **Enumeraciones claras** | Los nombres de las enumeraciones dejan claro cada estado. | Sí | EstadoVenta, EstadoPago, MetodoPago, TipoPrecio y EstadoDisponibilidad nombran cada estado con claridad. |
| **C-09** | **Constantes por significado** | Las constantes se nombran por lo que significan, no por su valor. | N/A | No hay constantes definidas. |

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **C-10** | **Elementos parecidos** | Los elementos parecidos se nombran de forma parecida. | Sí | Los atributos que se repiten tienen el mismo nombre en todas las clases (id, nombre, fecha, cantidad). |
| **C-11** | **Misma convención** | Todo el proyecto usa la misma convención de nombres. | Sí | Todo el diagrama respeta las mismas reglas de escritura: PascalCase para clases y enumeraciones, camelCase para atributos y métodos, y mayúsculas sostenidas para los valores de las enumeraciones. |
| **C-12** | **Sin abreviaturas innecesarias** | Se evitan las abreviaturas innecesarias y los nombres que se confunden fácil. | Sí | No se hace uso de abreviaturas confusas al momento de nombrar las variables. |
| **C-13** | **Abreviaturas documentadas** | Las abreviaturas que se usan están documentadas. | N/A | No se usan abreviaturas, así que no hace falta una tabla. |
| **C-14** | **Calificadores al final** | En los nombres de valores calculados, el calificador va al final. | Sí | En las funciones que realizan cálculos o evaluaciones, el concepto principal que se obtiene va ubicado al final del nombre (por ejemplo, calcularTotalPagado, calcularSaldoPendiente y determinarEstadoPago). |

## D. Tipos de datos básicos

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **D-01** | **Sin números mágicos** | No hay números mágicos en el código. | Sí | Se evitó fijar valores numéricos directamente en las funciones; parámetros como el umbral de existencias se obtienen de forma dinámica desde ConfiguracionNegocio. |
| **D-02** | **Constantes con nombre** | Los valores constantes importantes tienen nombre. | N/A | Se verifica en la implementación |

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **D-03** | **Compilación sin advertencias** | El código compila sin advertencias. | N/A | Se verifica al compilar |
| **D-04** | **Desbordamiento de enteros** | Se considera el desbordamiento de enteros. | N/A | Se verifica al compilar |
| **D-05** | **Precisión de decimales** | Se tienen en cuenta la precisión y el redondeo de los números decimales (float y double). | Sí | Se descartó el uso de tipos flotantes (float o double) en favor de BigDecimal, garantizando exactitud matemática en todos los datos numéricos de la aplicación |
| **D-06** | **Booleanos para sí o no** | Se usan booleanos para condiciones de sí o no. | Sí | Las variables booleanas se emplean únicamente para condiciones binarias (como activo). Los estados con tres o más opciones se manejan mediante enumeraciones. |
| **D-07** | **Enumeraciones para valores fijos** | Se usan enumeraciones cuando hay un conjunto fijo y conocido de valores. | Sí | Se utilizan cinco enumeraciones para conjuntos fijos de datos. Se identifica que la función generarReporte(tipo, periodo) también debería adoptar enumeraciones en futuras versiones si estos parámetros son cerrados. |
| **D-08** | **Dinero sin errores de precisión** | Los cálculos con dinero evitan los problemas de precisión de los decimales. | Sí | Todos los montos monetarios se definieron como BigDecimal para prevenir errores de redondeo o pérdida de centavo (en caso de que se manejen en el futuro ya que por ahora todo es a base de números enteros). |
| **D-09** | **Valores inválidos en enumeraciones** | Las pruebas con enumeraciones verifican valores inválidos. | No | No se definió cómo se prueban las enumeraciones con valores inválidos. Es parte de las pruebas que faltan por escribir, se implementará más adelante en el desarrollo del proyecto. |
| **D-10** | **Un tipo por cada dato que cambia** | Se usa un tipo distinto por cada clase de dato que puede cambiar. | No | El dinero se maneja como BigDecimal en todas las clases, sin un tipo propio. Se decidió no usar objetos de valor porque no es algo de lo que se tenga un dominio adecuado para ser aplicado en este momento. |

| ID | Aspecto por evaluar | Descripción | ¿Cumple? | Justificación |
|---|---|---|---|---|
| **D-11** | **Tipos del mundo real** | Los nombres de los tipos se orientan a las entidades reales que representan. | Sí | Los tipos modelan entidades reales: Producto, Venta, Pago, Cliente y de ahí se derivan otras que también ayudan a representar el problema a solucionar. |